



Design Patterns for Beginners

Presented By,
Md. Sayeed Rahman
Software Engineer III,
Astha IT Ltd.

Why reinvent the wheel someone has already invented for you?

Imagine you're building a bicycle. Would you:

- Design a new wheel from scratch?
- Figure out how gears work all by yourself?
- Invent brakes for the first time?

Of course not! You'd use existing wheels, gears, and brakes that people have perfected over decades.



You don't have to reinvent
the wheel.

The Same Goes for Code

Every problem you face, thousands of developers have faced before. Design patterns are their proven solutions – ready for you to use.





What Are Design Patterns?

Definition:

Design patterns are reusable solutions to common problems in software design.

Think of them as:

- Blueprints, not finished code
- Recipes, not cooked meals
- Templates that you adapt to your specific situation

The Best Part:

They come with a guarantee – used and tested by millions of developers worldwide.



Design Patterns Come in Three Flavors

1. Creational Patterns: How to create objects smartly

Examples: **Singleton, Factory, Builder**

2. Structural Patterns: How to organize and connect your code

Examples: **Adapter, Decorator, Facade**

3. Behavioral Patterns: How objects communicate and work together

Examples: **Observer, Strategy, Command**

The book **Gang of Four: Design Patterns: Elements of Reusable Object-Oriented Software** introduces 23 design patterns.



Common Misconceptions

Myth: "They're too advanced for beginners"

Reality: They're tools to make YOUR life easier, not harder

Myth: "I need to memorize all 23 patterns"

Reality: Start with 3-5 common ones, learn others as needed

Design patterns are easy to learn when you learn by understanding the problem first, and how the design pattern solves it.



Singleton Pattern: One Instance to Rule Them All

The Problem: You need exactly ONE instance of something across your entire app

- ONE database connection (not 10 wasting resources)
- ONE configuration manager (not conflicting configs)
- ONE logger (all logs in one place)

The Solution: Make the class control its own single instance

How It Works:

- Private constructor (can't create from outside)
- Static instance variable (holds the one copy)
- Public method returns same instance every time

Analogy: Like a country having ONE president



Imagine we don't want more than one db instance...

Title: Singleton in Action

✗ Messy Way:

```
csharp

// Multiple instances created everywhere!
var db1 = new DatabaseConnection();
var db2 = new DatabaseConnection();
var db3 = new DatabaseConnection();

// Now you have 3 connections wasting resources!
// db1 == db2? FALSE - Different objects!
```



✓ Singleton Way:

csharp

```
// Always get the same instance!
```

```
var db1 = DatabaseConnection.Instance;
```

```
var db2 = DatabaseConnection.Instance;
```

```
var db3 = DatabaseConnection.Instance;
```

```
// All three are the SAME object
```

```
// db1 == db2 == db3 → True!
```

```
// Use it anywhere in your app
```

```
DatabaseConnection.Instance.Connect();
```

```
DatabaseConnection.Instance.ExecuteNonQuery("SELECT * FROM Users");
```

When to Use: ✓ Database connections, Configuration managers, Logging systems, Cache managers



Factory Pattern: The Object Creation Expert

The Problem: You need to create different types of objects, but you don't know which type until runtime

Examples:

- Notification system: Email, SMS, or Push?
- Document generator: PDF, Word, or Excel?
- Payment processor: Credit Card, PayPal, or Crypto?

The Solution: Centralize all object creation in ONE place (the Factory)

Benefits:

- All creation logic in one spot
- Easy to add new types
- Client code stays clean

Factory Pattern: The Object Creation Expert

✗ Messy Way:

csharp

// This code scattered EVERYWHERE in your app!

```
INotifier notifier;
```

```
if (type == "email")
```

```
    notifier = new EmailNotifier();
```

```
else if (type == "sms")
```

```
    notifier = new SMSNotifier();
```

```
else if (type == "push")
```

```
    notifier = new PushNotifier();
```

```
notifier.Send("Hello");
```





✓ Factory Way:

csharp

```
// Clean! Factory handles creation internally
var notifier = NotificationFactory.Create("email");
notifier.Send("Hello");

// Want SMS instead? Just change the parameter
var smsNotifier = NotificationFactory.Create("sms");
smsNotifier.Send("Hello");

// Easy to switch at runtime
var type = userPreference; // "email", "sms", or "push"
var notifier = NotificationFactory.Create(type);
notifier.Send("Hello");
```

When to Use: ✓ Multiple object types, Runtime decisions, Complex creation logic



Observer Pattern: The Automatic Notification System

The Problem: One object changes, and many others need to know about it

Real Examples:

- YouTube: Creator uploads → ALL subscribers notified
- Stock market: Price changes → All traders alerted
- Email: New message → All devices updated

The Solution: Objects "subscribe" to events and get automatically notified when changes happen

Key Benefit: Loose coupling - the subject doesn't need to know WHO is watching!



Without observer, it's hard to notify everyone one by one, harder to add/remove peoples from list..

✗ Messy Way:

```
csharp
```

```
// Hard-coded notifications - tightly coupled!
```

```
channel.UploadVideo("New Tutorial");
```

```
alice.Notify("New Tutorial");
```

```
bob.Notify("New Tutorial");
```

```
charlie.Notify("New Tutorial");
```

```
// Want to add David? Modify the code!
```

```
// Want to remove Bob? Modify the code!
```

✓ Observer Way:

csharp

// Subscribe once, get notified automatically!

```
var channel = new YouTubeChannel();
```

```
channel.Subscribe(alice);
```

```
channel.Subscribe(bob);
```

```
channel.Subscribe(charlie);
```

// Upload video - ALL subscribers auto-notified!

```
channel.UploadVideo("Design Patterns Tutorial");
```

// Output:

// Alice got notification: Design Patterns Tutorial

// Bob got notification: Design Patterns Tutorial

// Charlie got notification: Design Patterns Tutorial

// Easy to add/remove subscribers

```
channel.Subscribe(david);    // David added
```

```
channel.Unsubscribe(bob);   // Bob removed
```

Builder Pattern: Complex Objects Creation in Steps

The Problem: Creating objects with many optional parameters is messy and confusing

Examples:

- Building a User: name, email, phone, address, age, gender, preferences...
- Creating a Pizza: size, crust, sauce, cheese, 10+ toppings...
- Configuring a Car: model, color, engine, wheels, seats, sunroof, GPS...

Without Builder:

```
csharp

// Telescoping constructors - nightmare!
new User("John", "john@email.com", "123-456", "123 Main St", 25, "M", true, false, "
// What does true, false, "dark" mean? 🤔
```

The Solution: Build objects step by step using a fluent, readable interface

No one can remember 50 params in constructor

✗ Messy Way:

csharp

// Constructor hell - confusing parameter order

```
var user1 = new User("John", "john@email.com", null, null, 0, null);  
var user2 = new User("Jane", "jane@email.com", "123-456", null, 25, "F");  
var user3 = new User("Bob", "bob@email.com", "789-012", "123 Main", 30, "M");
```

// What's null? What's 0? Hard to read!

// Or multiple constructors (constructor overload hell)

```
var user4 = new User("Alice", "alice@email.com");  
var user5 = new User("Tom", "tom@email.com", "555-1234");  
// Need 20 constructors for all combinations! 🤖
```

Builder comes to the rescue!

✓ Builder Way:

csharp

// Clear, readable, fluent interface

```
var user1 = new UserBuilder()  
    .WithName("John")  
    .WithEmail("john@email.com")  
    .Build();
```

```
var user2 = new UserBuilder()  
    .WithName("Jane")  
    .WithEmail("jane@email.com")  
    .WithPhone("123-456")  
    .WithAge(25)  
    .WithGender("F")  
    .Build();
```

Adapter Pattern: Making Incompatible Interfaces Work Together

The Problem:

You have two classes that should work together but have incompatible interfaces

Real-World Analogy:

US plug (2 prongs) won't fit European socket (2 round pins)

Solution? Use an adapter!



Without Adapter, Bad code looks like this

```
// But third-party library uses different names
var stripePayment = new StripePaymentService();
stripePayment.MakePayment(100); // Different method name!

var paypalPayment = new PayPalAPI();
paypalPayment.SendMoney(50); // Different method name!

// Your code breaks! Can't use them interchangeably!
// You'd have to write if-else everywhere:
if (paymentType == "stripe")
    stripePayment.MakePayment(amount);
else if (paymentType == "paypal")
    paypalPayment.SendMoney(amount);
```

With adapter, it's better!

✓ Adapter Way:

csharp

// Create adapters that implement your interface

```
var stripeAdapter = new StripeAdapter();
```

```
var paypalAdapter = new PayPalAdapter();
```

// Now use them consistently!

```
IPaymentProcessor payment1 = stripeAdapter;
```

```
payment1.ProcessPayment(100);
```

// Internally calls: stripePayment.MakePayment(100)

```
IPaymentProcessor payment2 = paypalAdapter;
```

```
payment2.ProcessPayment(50);
```

// Internally calls: paypalPayment.SendMoney(50)



Decorator Pattern: Extend What You Can't Modify (third party libraries, SDKs, APIs)

The Problem: You're using BkashAPI (third-party payment library) but need extra features:

- Want to log every transaction
- Want to send notifications after payment
- Want to validate before processing

But You Can't Modify BkashAPI Because: ❌ It's a third-party NuGet package ❌ You don't own the source code ❌ Updates would overwrite your changes ❌ You'd break their code

The Solution: Create decorator classes that wrap BkashAPI and add your features while keeping the same interface

Without decorators

✗ Messy Way:

```
csharp

// Using BkashAPI directly everywhere
var bkash = new BkashAPI();

// Scattered throughout your code:
bkash.ProcessPayment(100);
// Log manually every time
Console.WriteLine("Payment processed");
// Send notification manually every time
SendSMS("Payment successful");

bkash.ProcessPayment(200);
// Forgot to log! 🤖
// Forgot notification! 🤖
```

With decorators

✓ Decorator Way:

```
csharp

// Wrap BkashAPI with decorators
IPaymentService payment = new BkashAPI();
payment = new LoggingDecorator(payment);
payment = new NotificationDecorator(payment);

// Usage - Clean and automatic!
payment.ProcessPayment(100);

// Output:
// [LOG] Processing payment: $100
// [BkashAPI] Payment processed: $100
// [NOTIFICATION] SMS sent: Payment of $100 successful
```



Quick Reference: Your 7 Essential Patterns

Pattern	The Problem	The Solution	Quick Win
Singleton	Multiple instances waste resources	One controlled instance	Database connections, configs
Factory	<code>if-else</code> hell for object creation	Centralized creation	Notification systems, documents
Observer	Hard-coded notifications	Subscribe and auto-notify	Event systems, live updates
Strategy	Giant conditionals for algorithms	Swappable behavior objects	Payment methods, sorting
Decorator	100 subclasses for combinations	Stack wrappers	Coffee orders, file streams
Builder	Constructor with 10+ parameters	Fluent step-by-step building	User profiles, complex configs
Adapter	Incompatible interfaces	Wrapper that translates	Third-party APIs, legacy code



Revise other patterns if you feel enthusiastic.

Thank you.